Exploratory Data Analysis

EDA Theory Project

Air Quality Analysis and Next-Day PM2.5 Forecasting

Dongsi Monitoring Station, Beijing (2013 – 2017)

Submitted by

Harshit Arora (23BDS0002)

School: SCOPE

Under the guidance of

Dr. Balaji G.N.

VIT, Vellore



Abstract

This project analyses four years (March 2013 to February 2017) of hourly air-quality and weather records from the Dongsi monitoring station in central Beijing, taken from the Kaggle Beijing Multi-Site Air-Quality dataset, and builds a model that forecasts the next day's average PM2.5 concentration. After cleaning the data (time-based interpolation of up to 9% missing values per column and removal of 18 invalid ozone readings), the exploratory analysis shows that PM2.5 is strongly right-skewed, that 43.5% of days exceed China's 24-hour standard of 75 µg/m³, and that pollution is worst in winter, at night, under easterly and southerly winds and on calm days. Daily PM2.5 has a short memory: the lag-1 autocorrelation is 0.55 and it fades within about three days. A Random Forest that uses today's PM2.5 and tomorrow's weather forecasts next-day PM2.5 with a mean absolute error of 31.2 µg/m³ and R^2 of 0.684 on the final year of data, compared with R^2 of 0.175 for a naive 'tomorrow equals today' forecast, and correctly warns of 85.1% of polluted days.

Contents

Abstract....2
1. Introduction....3
2. Dataset Description....4
3. Tools and Methodology....5
4. Data Loading and Preparation....6
5. Data Quality: Missing Values and Outliers....8
6. Univariate Analysis....12
7. Bivariate Analysis....14
8. Multivariate Analysis....18
9. Time Series Analysis....19
10. Forecasting Model....22
11. Key Findings....27
12. Conclusion, Limitations and Future Work....28
References....29
Appendix A: Complete Python Code....30

1. Introduction

1.1 Background

PM2.5 refers to airborne particles smaller than 2.5 micrometres. They are small enough to reach deep into the lungs and the bloodstream, which makes PM2.5 the air pollutant most closely linked to heart and lung disease. Beijing has been one of the most studied examples of severe urban air pollution, with winter haze episodes during which PM2.5 rises to several hundred micrograms per cubic metre. For comparison, the World Health Organization's 2021 guideline is 15 µg/m³ for a 24-hour average and 5 µg/m³ for an annual average, and China's own 24-hour standard (Grade II) is 75 µg/m³.

Because pollution levels change quickly with the weather, a reliable short-term forecast lets residents, schools and hospitals plan ahead. This project uses exploratory data analysis (EDA) to understand what drives PM2.5 at one Beijing station and then uses those findings to build a next-day forecasting model.

1.2 Objectives

1. Inspect and clean four years of hourly data, handling missing values and invalid sensor readings appropriately.
2. Describe the distribution of PM2.5 and how often the air is polluted.
3. Identify when (season and hour of day) and under which weather conditions PM2.5 is highest.
4. Examine the time-series structure of daily PM2.5: trend, seasonality and autocorrelation.
5. Build and evaluate a model that forecasts the next day's average PM2.5.

2. Dataset Description

2.1 Source

The data come from the Kaggle dataset *Beijing Multi-Site Air-Quality Data Set* (kaggle.com/datasets/sid321axn/beijing-multisite-airquality-data-set), which republishes the UCI Machine Learning Repository dataset created by Song Chen (2017). It contains hourly measurements from 12 nationally controlled monitoring sites in Beijing. The pollutant readings come from the Beijing Municipal Environmental Monitoring Center, and each site is matched with weather data from the nearest China Meteorological Administration station. The dataset is released under the CC BY 4.0 licence.

This project uses one station, **Dongsi**, an urban site in central Beijing (Dongcheng district). The file PRSA_Data_Dongsi_20130301-20170228.csv holds **35,064 hourly records and 18 columns**, covering 1 March 2013 00:00 to 28 February 2017 23:00.

2.2 Variables

Column	Description	Unit / type
No	Row number	integer
year, month, day, hour	Date and hour of the reading	integer
PM2.5	Fine particulate matter (diameter ≤ 2.5 μm)	μg/m³
PM10	Particulate matter (diameter ≤ 10 μm)	μg/m³
SO2	Sulphur dioxide	μg/m³
NO2	Nitrogen dioxide	μg/m³
CO	Carbon monoxide	μg/m³
O3	Ozone	μg/m³
TEMP	Air temperature	°C
PRES	Air pressure	hPa
DEWP	Dew point temperature (a measure of humidity)	°C
RAIN	Precipitation during the hour	mm
wd	Wind direction (16 compass points)	categorical
WSPM	Wind speed	m/s
station	Name of the monitoring station	categorical

Table 1: Columns of the Dongsi station file

2.3 Why this dataset

The dataset is far less commonly used than standard teaching datasets, and it suits a complete EDA-to-model project. It contains real data-quality problems (missing values with long gaps, and impossible sensor readings), a mix of numerical and categorical variables, a clear time structure with daily and yearly cycles, and a practical prediction target: tomorrow's air quality.

3. Tools and Methodology

3.1 Tools

The analysis was carried out in Python 3.13 inside a JupyterLab 4.6 notebook (EDA_Theory_Project.ipynb). The libraries used are pandas 3.0 and NumPy 2.5 for data handling, Matplotlib 3.11 and seaborn 0.13 for visualisation, statsmodels 0.15 for time-series decomposition and autocorrelation, and scikit-learn 1.9 for the forecasting models. Figure 1 shows the notebook open in JupyterLab; the remaining figures are screenshots of the executed notebook cells, each showing the code with its output.

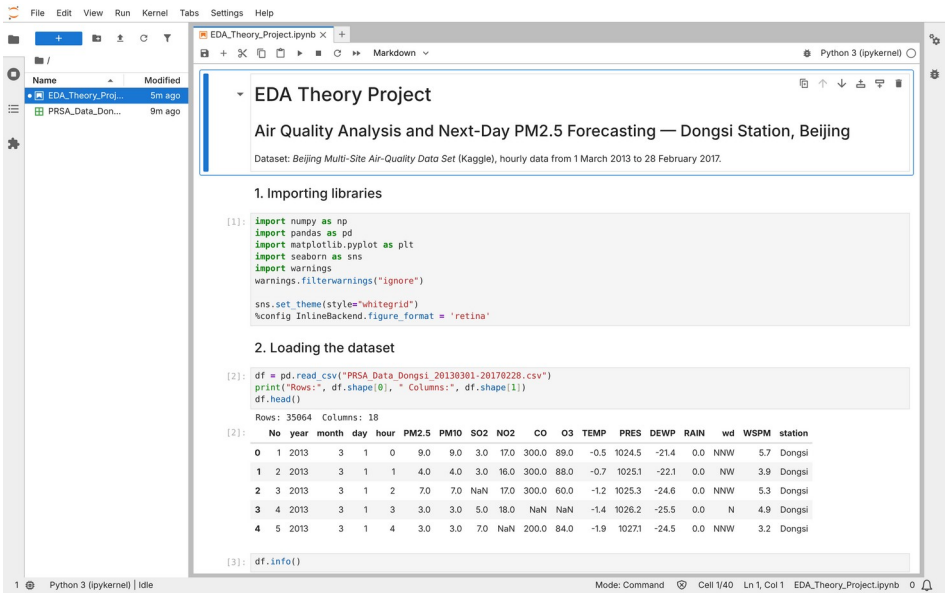


Figure 1: The project notebook open in JupyterLab

3.2 Workflow

Only the techniques needed to answer the project's question were used. Clustering and principal component analysis were not applied, because the goal is to forecast a single target variable from a small set of features that are already easy to interpret. The analysis followed these steps:

1. Load the CSV file and inspect its structure.
2. Prepare the data: build a datetime index, drop redundant columns, check duplicates and time continuity.
3. Assess data quality: measure missing values and gap lengths, detect outliers with box plots and the IQR rule, and identify invalid sensor readings.
4. Handle the problems: remove invalid readings, interpolate numeric gaps in time and forward-fill wind direction.
5. Univariate analysis: summary statistics, skewness and kurtosis, the PM2.5 distribution and the number of days in each air-quality level.
6. Bivariate and multivariate analysis: PM2.5 by month, hour, season and wind direction, the correlation matrix, and the combined effect of humidity and wind.
7. Time series analysis: daily trend, seasonal decomposition, ACF and PACF.
8. Modelling: engineer features, split the data chronologically, compare a baseline, Linear Regression and Random Forest, and evaluate the best model as a pollution warning.

4. Data Loading and Preparation

4.1 Importing libraries

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

sns.set_theme(style="whitegrid")
%config InlineBackend.figure_format = 'retina'
```

Figure 2: Importing libraries

pandas and NumPy handle the data, Matplotlib and seaborn draw the plots, and statsmodels and scikit-learn are imported later in the cells where they are needed. The retina setting makes inline figures render at double resolution so that they stay sharp.

4.2 Loading the dataset

[2]:
```
df = pd.read_csv("PRSA_Data_Dongsi_20130301-20170228.csv")
print("Rows:", df.shape[0], " Columns:", df.shape[1])
df.head()
```

```
Rows: 35064  Columns: 18
```

[2]:

	No	year	month	day	hour	PM2.5	PM10	SO2	NO2	CO	O3	TEMP	PRES	DEWP	RAIN	wd	WSPM	station
0	1	2013	3	1	0	9.0	9.0	3.0	17.0	300.0	89.0	-0.5	1024.5	-21.4	0.0	NNW	5.7	Dongsi
1	2	2013	3	1	1	4.0	4.0	3.0	16.0	300.0	88.0	-0.7	1025.1	-22.1	0.0	NW	3.9	Dongsi
2	3	2013	3	1	2	7.0	7.0	NaN	17.0	300.0	60.0	-1.2	1025.3	-24.6	0.0	NNW	5.3	Dongsi
3	4	2013	3	1	3	3.0	3.0	5.0	18.0	NaN	NaN	-1.4	1026.2	-25.5	0.0	N	4.9	Dongsi
4	5	2013	3	1	4	3.0	3.0	7.0	NaN	200.0	84.0	-1.9	1027.1	-24.5	0.0	NNW	3.2	Dongsi

Figure 3: Loading the CSV file and displaying the first rows

The file loads with 35,064 rows and 18 columns. Each row is one hour, starting at midnight on 1 March 2013. Missing values (NaN) already appear in the first five rows in SO2, NO2, CO and O3, so missing data will need attention.

[3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 35064 entries, 0 to 35063
Data columns (total 18 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   No       35064 non-null  int64  
 1   year     35064 non-null  int64  
 2   month    35064 non-null  int64  
 3   day      35064 non-null  int64  
 4   hour     35064 non-null  int64  
 5   PM2.5    34314 non-null  float64
 6   PM10     34511 non-null  float64
 7   SO2      34401 non-null  float64
 8   NO2      33463 non-null  float64
 9   CO       31867 non-null  float64
 10  O3       34400 non-null  float64
 11  TEMP     35044 non-null  float64
 12  PRES     35044 non-null  float64
 13  DEWP     35044 non-null  float64
 14  RAIN     35044 non-null  float64
 15  wd       34986 non-null  str    
 16  WSPM     35050 non-null  float64
 17  station  35064 non-null  str    
dtypes: float64(11), int64(5), str(2)
memory usage: 4.8 MB
```

Figure 4: Structure and data types (info)

There are 11 floating-point columns (pollutants and weather), 5 integer columns (the row number and the four date parts) and 2 text columns (wd and station). Twelve columns have fewer than 35,064 non-null values, confirming missing data. The date is spread over four separate integer columns, so it has to be combined into a single datetime before any time-based analysis.

4.3 Preparing the data

[4]:
```
print("Stations in file:", df["station"].unique().tolist())

# build one datetime column from year, month, day and hour
df["datetime"] = pd.to_datetime(df[["year", "month", "day", "hour"]])
df = df.set_index("datetime")
df = df.drop(columns=["No", "year", "month", "day", "hour", "station"])

# check duplicates and whether every hour is present
full_range = pd.date_range(df.index.min(), df.index.max(), freq="h")
print("Duplicate rows       :", df.duplicated().sum())
print("Duplicate timestamps :", df.index.duplicated().sum())
print("Expected hours       :", len(full_range), " Rows present:", len(df))
```

```
Stations in file: ['Dongsi']
Duplicate rows       : 0
Duplicate timestamps : 0
Expected hours       : 35064  Rows present: 35064
```

Figure 5: Building the datetime index and checking duplicates and continuity

The file contains only one station, so the station column carries no information and is dropped, together with the row number and the four date parts once they have been combined into a datetime index. There are no duplicate rows and no duplicate timestamps, and the number of rows (35,064) equals the number of hours between the first and last timestamp. The timeline is therefore complete: missing data appear only as empty values, never as missing hours.

5. Data Quality: Missing Values and Outliers

5.1 Missing values

[5]:
```
def longest_gap(s):
    # longest run of consecutive missing hours in a column
    runs = s.isna().astype(int).groupby(s.notna().cumsum()).sum()
    return runs.max()

missing = pd.DataFrame({
    "missing_count": df.isna().sum(),
    "missing_pct": (df.isna().mean() * 100).round(2),
    "longest_gap_hours": [longest_gap(df[c]) for c in df.columns]
})
missing.sort_values("missing_pct", ascending=False)
```

[5]:

	missing_count	missing_pct	longest_gap_hours
CO	3197	9.12	1517
NO2	1601	4.57	900
PM2.5	750	2.14	58
SO2	663	1.89	57
O3	664	1.89	57
PM10	553	1.58	57
wd	78	0.22	5
TEMP	20	0.06	6
PRES	20	0.06	6
DEWP	20	0.06	6
RAIN	20	0.06	6
WSPM	14	0.04	5

Figure 6: Missing values and longest gap per column

CO has the most missing values (3,197 hours, 9.12%), followed by NO2 (1,601 hours, 4.57%) and PM2.5 (750 hours, 2.14%). The weather columns are almost complete, each with less than 0.1% missing. The longest-gap column matters as much as the percentage: CO has one gap of 1,517 consecutive hours (about 63 days) and NO2 one of 900 hours (37.5 days), whereas PM2.5 is never missing for more than 58 hours in a row and the weather variables for no more than 6.

```
[6]: miss = missing[missing["missing_count"] > 0].sort_values("missing_pct")
plt.figure(figsize=(9, 4))
plt.barh(miss.index, miss["missing_pct"], color="indianred")
for i, v in enumerate(miss["missing_pct"]):
    plt.text(v + 0.1, i, f"{v}%", va="center", fontsize=9)
plt.title("Percentage of Missing Values per Column")
plt.xlabel("Missing (%)")
plt.show()
```

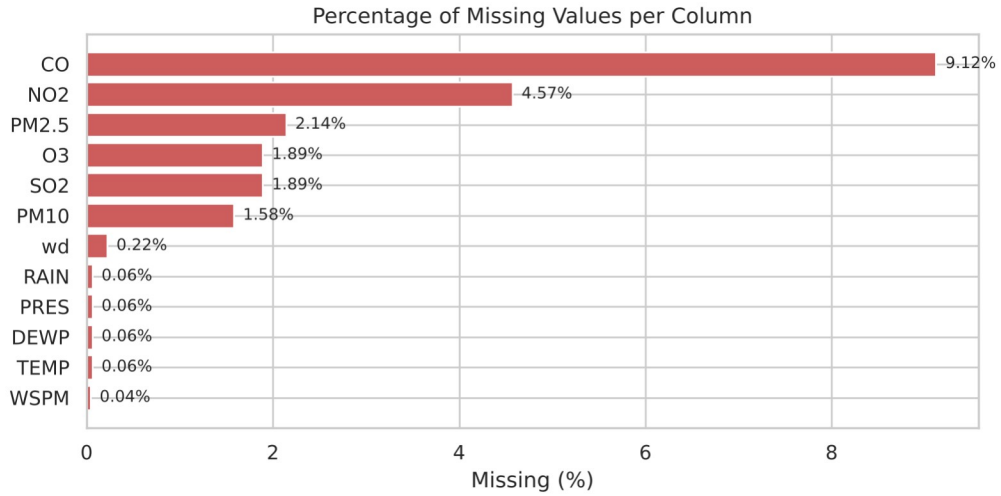


Figure 7: Percentage of missing values per column

The bar chart shows the same pattern: missing data are concentrated in the gas analysers (CO and NO2), while the particle and weather measurements are mostly complete. Since the target variable PM2.5 has only short gaps, it can be filled reliably.

5.2 Outliers and invalid readings

```
[7]: pollutants = ["PM2.5", "PM10", "SO2", "NO2", "CO", "O3"]
fig, axes = plt.subplots(1, 6, figsize=(15, 4))
for ax, col in zip(axes, pollutants):
    sns.boxplot(y=df[col], ax=ax, color="lightsteelblue")
    ax.set_title(col)
    ax.set_ylabel("")
fig.suptitle("Box Plots of Pollutant Concentrations (µg/m³)")
plt.tight_layout()
plt.show()
```

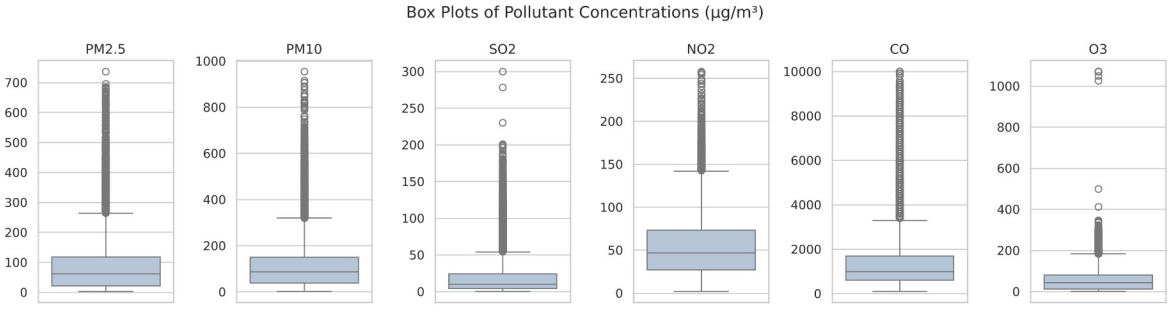


Figure 8: Box plots of the six pollutants

Every pollutant is right-skewed, with many points above the upper whisker. For most pollutants these points form a continuous tail, which is what real pollution episodes look like. Ozone (O3) is different: a few values above 1,000 µg/m³ sit far away from the rest of the data, which suggests a measurement problem rather than real pollution.

[8]:
```
rows = []
for col in pollutants:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    upper = q3 + 1.5 * iqr
    rows.append([col, q1, q3, iqr, upper, (df[col] > upper).sum(), df[col].max()])

pd.DataFrame(rows, columns=["variable", "Q1", "Q3", "IQR", "upper_fence",
                            "n_outliers", "max"]).round(1)
```

[8]:

	variable	Q1	Q3	IQR	upper_fence	n_outliers	max
0	PM2.5	22.0	119.0	97.0	264.5	1622	737.0
1	PM10	38.0	151.0	113.0	320.5	1335	955.0
2	SO2	4.0	24.0	20.0	54.0	2606	300.0
3	NO2	27.0	73.0	46.0	142.0	600	258.0
4	CO	600.0	1700.0	1100.0	3350.0	1976	10000.0
5	O3	12.0	81.0	69.0	184.5	1318	1071.0

Figure 9: Outlier counts using the IQR rule

The IQR rule (values above Q3 + 1.5 × IQR) flags 1,622 PM2.5 hours above 264.5 µg/m³ and 2,606 SO2 hours above 54 µg/m³. No low outliers exist, because every lower fence is below zero. These flagged PM2.5 values are real winter haze episodes, not errors, so they are **kept**: removing them would delete exactly the events the forecast needs to predict.

[9]:
```
high_o3 = df[df["O3"] > 400]
print("O3 readings above 400:", len(high_o3))
print("Dates:", sorted(set(high_o3.index.date)))
df.loc["2013-08-15 20:00":"2013-08-16 03:00", ["NO2", "O3", "TEMP"]]
```

```
O3 readings above 400: 18
Dates: [datetime.date(2013, 8, 15), datetime.date(2013, 8, 16), datetime.date(2013, 8, 17), datetime.date(2017, 1,
6)]
```

[9]:

datetime	NO2	O3	TEMP
2013-08-15 20:00:00	35.0	104.5	31.1
2013-08-15 21:00:00	37.0	104.5	30.9
2013-08-15 22:00:00	45.0	2.0	30.7
2013-08-15 23:00:00	40.0	1071.0	29.5
2013-08-16 00:00:00	34.0	1071.0	27.2
2013-08-16 01:00:00	28.0	413.0	27.2
2013-08-16 02:00:00	19.0	1050.0	26.7
2013-08-16 03:00:00	30.0	NaN	26.6

Figure 10: Investigating the extreme ozone readings

Eighteen ozone readings exceed 400 µg/m³, on 15–17 August 2013 and on 6 January 2017. In August 2013 ozone drops from 104.5 to 2.0, jumps to 1,071 and then swings between 413 and 1,050 within a few hours, late at night. Ozone forms in sunlight, so such values at night are not physically possible. The January 2017 reading also jumps, from 15 to 500 µg/m³ within one hour on a cold winter afternoon (4.9 °C). These readings are sensor errors.

[10]:
```
# O3 jumps between 2 and 1071 within a few hours -> sensor error, mark as missing
df.loc[df["O3"] > 400, "O3"] = np.nan
print("O3 maximum after removing invalid readings:", df["O3"].max())
```

```
O3 maximum after removing invalid readings: 346.0
```

Figure 11: Removing the invalid ozone readings

The 18 invalid readings are set to missing so that they will be interpolated with the other gaps. The highest remaining value, 346 µg/m³, is the hour just after the January 2017 spike and may also be affected; every other reading above 300 µg/m³ occurs on a hot summer afternoon (29–35 °C), which is

exactly when real ozone peaks form. Because ozone is not used in the forecasting model, this single borderline reading is kept.

5.3 Handling missing values

[11]:
```
num_cols = df.select_dtypes("number").columns
pm_before = df["PM2.5"].describe()

# numeric columns: time-based linear interpolation between neighbouring hours
df[num_cols] = df[num_cols].interpolate(method="time")
# wind direction (categorical): carry the last observed direction forward
df["wd"] = df["wd"].ffill()

print("Missing values left:", df.isna().sum().sum())
pd.DataFrame({"PM2.5 before": pm_before, "PM2.5 after": df["PM2.5"].describe()}).round(2)
```

```
Missing values left: 0
```

[11]:

	PM2.5 before	PM2.5 after
count	34314.00	35064.00
mean	86.19	86.14
std	86.58	86.26
min	3.00	3.00
25%	22.00	23.00
50%	61.00	61.00
75%	119.00	119.00
max	737.00	737.00

Figure 12: Time-based interpolation and forward filling

Numeric columns are filled with **time-based linear interpolation**, which estimates each missing hour from the readings on either side. This suits hourly data because pollution and weather change gradually from one hour to the next. Filling with the overall mean or median would be wrong here: it would insert an average value of about 86 µg/m³ into a clean summer night or into the middle of a winter haze episode. Wind direction is categorical, so each missing value is filled with the last observed direction (forward fill); its gaps are at most 5 hours long.

No missing values remain. The PM2.5 mean changes only from 86.19 to 86.14 µg/m³, the standard deviation from 86.58 to 86.26, and the median stays at 61, so the filling did not distort the distribution. One limitation is that the long CO and NO2 gaps are bridged by straight lines; these two variables are not used in the forecasting model, so this mainly affects the correlation matrix in Section 7.

6. Univariate Analysis

[12]:
```
key = ["PM2.5", "PM10", "NO2", "CO", "O3", "TEMP", "DEWP", "WSPM"]
stats = df[key].describe().T
stats["skewness"] = df[key].skew()
stats["kurtosis"] = df[key].kurt()
stats.round(2)
```

[12]:

	count	mean	std	min	25%	50%	75%	max	skewness	kurtosis
PM2.5	35064.0	86.14	86.26	3.00	23.0	61.0	119.0	737.0	1.99	5.71
PM10	35064.0	110.35	98.24	2.00	38.0	86.0	150.0	955.0	1.96	6.28
NO2	35064.0	53.95	34.21	2.00	27.0	47.0	74.0	258.0	1.01	1.10
CO	35064.0	1331.91	1169.04	100.00	600.0	1000.0	1700.0	10000.0	2.52	9.34
O3	35064.0	57.23	54.07	0.64	13.0	45.0	82.0	346.0	1.33	1.68
TEMP	35064.0	13.67	11.46	-16.80	3.1	14.6	23.5	41.1	-0.10	-1.17
DEWP	35064.0	2.45	13.81	-35.30	-8.8	3.0	15.0	28.8	-0.20	-1.11
WSPM	35064.0	1.86	1.28	0.00	1.0	1.5	2.4	10.5	1.57	3.37

Figure 13: Summary statistics with skewness and kurtosis

The mean PM2.5 is 86.14 µg/m³ while the median is only 61, and the skewness is 1.99. The distribution is therefore strongly right-skewed: most hours are moderate but some are extremely high (up to 737 µg/m³). The kurtosis of 5.71 confirms a heavy tail. CO is the most skewed variable (skewness 2.52, kurtosis 9.34). Temperature and dew point are almost symmetric (skewness −0.10 and −0.20) with negative kurtosis, meaning flat, spread-out distributions; this reflects Beijing's climate, which swings from −16.8 °C in winter to 41.1 °C in summer. The average wind speed is light, at 1.86 m/s.

[13]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
sns.histplot(df["PM2.5"], bins=60, kde=True, ax=axes[0], color="steelblue")
axes[0].set_title("Hourly PM2.5")
axes[0].set_xlabel("PM2.5 (µg/m³)")
sns.histplot(np.log1p(df["PM2.5"]), bins=60, kde=True, ax=axes[1], color="darkorange")
axes[1].set_title("log(1 + PM2.5)")
axes[1].set_xlabel("log(1 + PM2.5)")
plt.tight_layout()
plt.show()
```

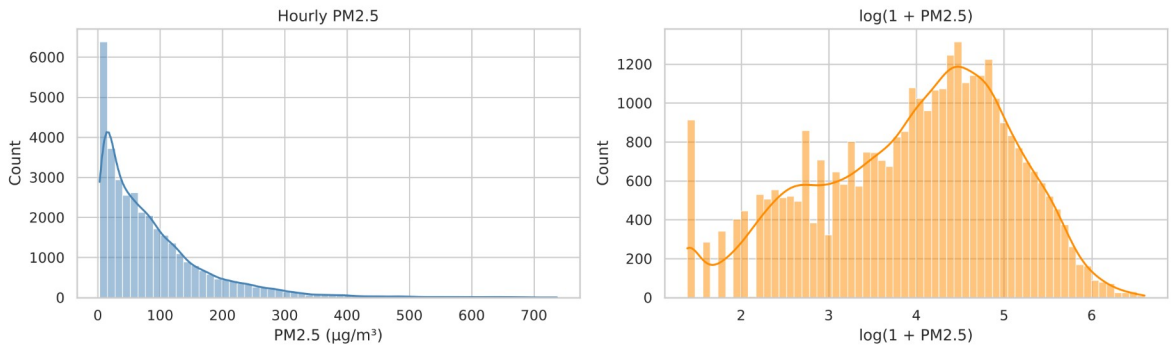


Figure 14: Distribution of hourly PM2.5 before and after log transformation

The histogram of raw PM2.5 has a tall peak at low values and a long tail stretching to 737 µg/m³. After the log(1 + x) transformation the distribution is much closer to symmetric, which would help a linear model; tree-based models such as Random Forest do not need it. The isolated bar at the far left of the log plot comes from 892 readings of exactly 3 µg/m³ in the raw data, the lowest value the instrument reports.

[14]:
```
# daily average and China's AQI levels for 24-hour PM2.5
daily = df.resample("D").mean(numeric_only=True)

bins = [0, 35, 75, 115, 150, 250, np.inf]
levels = ["Excellent", "Good", "Lightly polluted", "Moderately polluted",
          "Heavily polluted", "Severely polluted"]
daily["aqi_level"] = pd.cut(daily["PM2.5"], bins=bins, labels=levels)

freq = daily["aqi_level"].value_counts().reindex(levels)
aqi_table = pd.DataFrame({"days": freq, "percent": (freq / len(daily) * 100).round(1)})
print("Number of days:", len(daily))
aqi_table
```

```
Number of days: 1461
```

[14]:

aqi_level	days	percent
Excellent	399	27.3
Good	426	29.2
Lightly polluted	271	18.5
Moderately polluted	157	10.7
Heavily polluted	144	9.9
Severely polluted	64	4.4

Figure 15: Daily mean PM2.5 grouped into China's AQI levels

The 35,064 hourly readings are averaged into 1,461 daily values and grouped using the PM2.5 breakpoints of China's Air Quality Index (35, 75, 115, 150 and 250 µg/m³). Only 399 days (27.3%) were Excellent and 426 (29.2%) Good. 636 days (43.5%) exceeded China's 24-hour standard of 75 µg/m³, and 208 days (14.2%) were Heavily or Severely polluted.

[15]:
```
colors = ["#2e7d32", "#9ccc65", "#fdd835", "#fb8c00", "#e53935", "#6a1b9a"]
plt.figure(figsize=(10, 4))
bars = plt.bar(aqi_table.index, aqi_table["days"], color=colors)
plt.bar_label(bars, labels=[f"{p}%" for p in aqi_table["percent"]])
plt.title("Number of Days in Each AQI Level (Daily Mean PM2.5)")
plt.ylabel("Days")
plt.xticks(rotation=15)
plt.show()
```

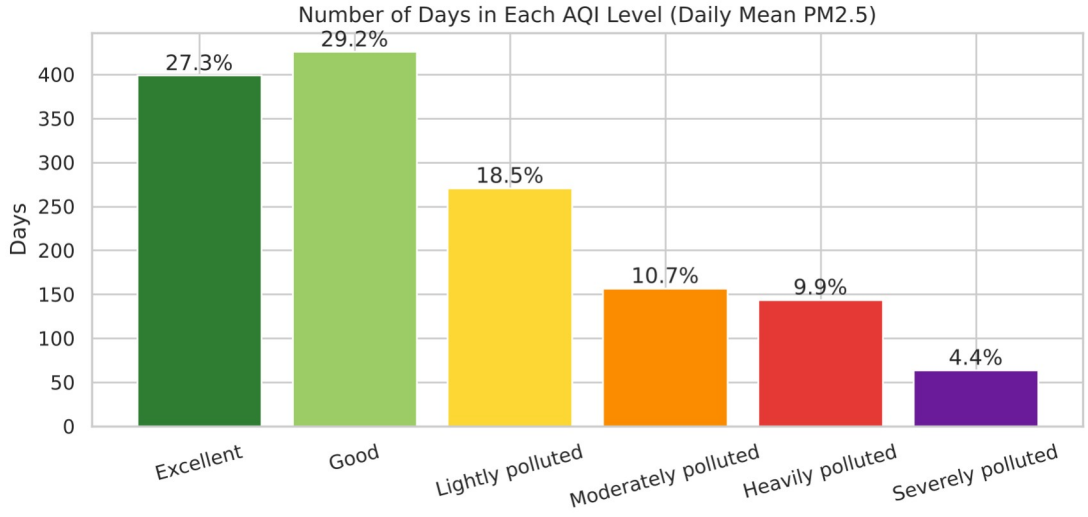


Figure 16: Number of days in each AQI level

The bar chart makes the same point visually: although the two cleanest levels are the most common, almost half of all days fall into one of the polluted levels, and a noticeable group of 64 days (4.4%) were Severely polluted, with daily averages above 250 µg/m³.

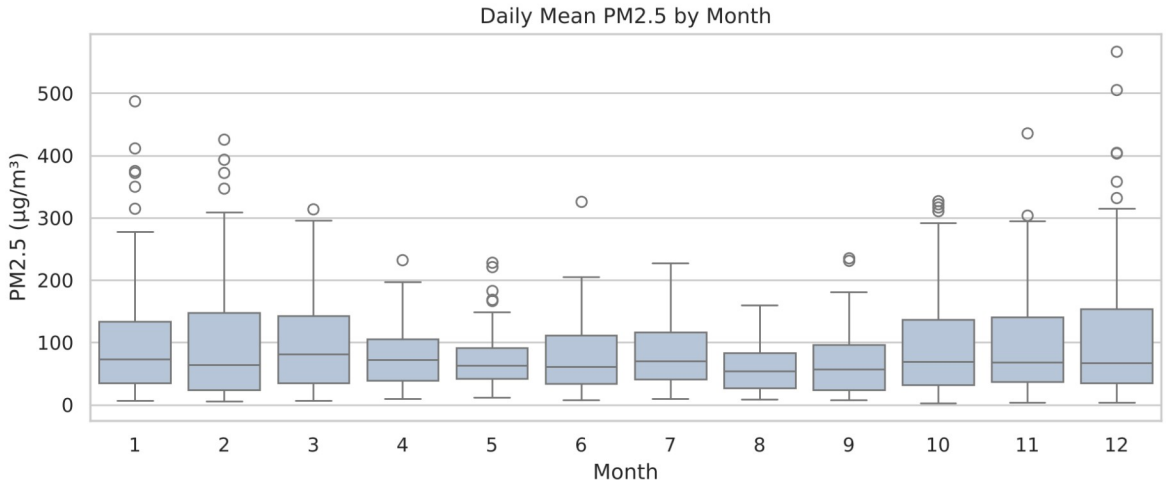
7. Bivariate Analysis

7.1 PM2.5 and time of year

[16]:
```
season_map = {12: "Winter", 1: "Winter", 2: "Winter", 3: "Spring", 4: "Spring", 5: "Spring",
              6: "Summer", 7: "Summer", 8: "Summer", 9: "Autumn", 10: "Autumn", 11: "Autumn"}
daily["month"] = daily.index.month
daily["season"] = daily["month"].map(season_map)

plt.figure(figsize=(11, 4))
sns.boxplot(data=daily, x="month", y="PM2.5", color="lightsteelblue")
plt.title("Daily Mean PM2.5 by Month")
plt.xlabel("Month")
plt.ylabel("PM2.5 (µg/m³)")
plt.show()

daily.groupby("season")["PM2.5"].agg(["mean", "median", "std", "max"]).round(1)
```



[16]:

season	mean	median	std	max
Autumn	88.2	65.3	75.3	435.6
Spring	82.4	71.5	59.7	313.9
Summer	71.2	61.8	46.5	325.7
Winter	103.1	67.6	99.8	567.4

Figure 17: Daily PM2.5 by month and summary by season

The monthly medians are fairly similar (roughly 55–80 µg/m³), but the spread changes a lot. In the winter months (December to February) and in October and November, the boxes and whiskers stretch much higher and there are many extreme days, while August is the cleanest month. The seasonal table confirms this: winter has the highest mean (103.1 µg/m³) and by far the largest standard deviation (99.8) and maximum (567.4), yet its median (67.6) is close to the other seasons. Winter pollution therefore comes from **occasional severe episodes** rather than consistently higher levels. These episodes coincide with Beijing's heating season (mid-November to mid-March), when coal burning adds emissions and cold, stable air traps them near the ground.

7.2 PM2.5 and hour of day

[17]:
```
df["hour"] = df.index.hour
df["season"] = df.index.month.map(season_map)

hourly_profile = df.groupby(["season", "hour"])["PM2.5"].mean().reset_index()
plt.figure(figsize=(10, 4))
sns.lineplot(data=hourly_profile, x="hour", y="PM2.5", hue="season",
             hue_order=["Winter", "Spring", "Summer", "Autumn"], marker="o")
plt.title("Average PM2.5 by Hour of Day and Season")
plt.xlabel("Hour of day")
plt.ylabel("Mean PM2.5 (µg/m³)")
plt.xticks(range(0, 24, 2))
plt.show()
```

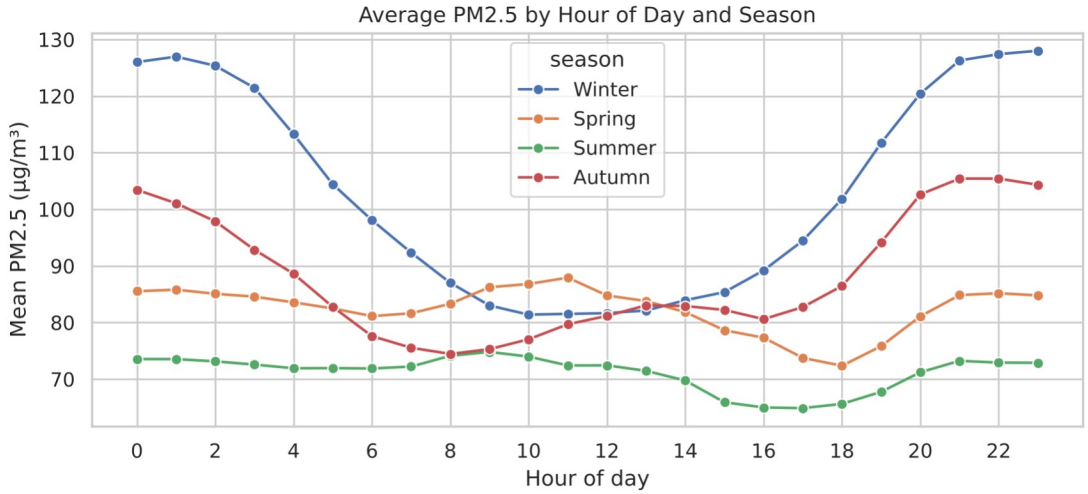


Figure 18: Average PM2.5 by hour of day for each season

In winter, PM2.5 is highest at night, peaking at 128 µg/m³ at 23:00, and lowest in late morning (81.4 µg/m³ at 10:00). Autumn follows the same pattern (peak 105.4 at 21:00). At night the lowest layer of the atmosphere becomes shallow and stable, so emissions from heating and traffic accumulate; during the day the sun heats the ground, the mixing layer deepens and pollution is diluted. Summer shows the weakest daily cycle, with values between about 65 and 75 µg/m³ and a minimum at 17:00, because there is no heating demand and strong daytime mixing.

7.3 PM2.5 and wind direction

[18]:
```
compass = ["N", "NNE", "NE", "ENE", "E", "ESE", "SE", "SSE",
           "S", "SSW", "SW", "WSW", "W", "WNW", "NW", "NNW"]
wind = df.groupby("wd")["PM2.5"].agg(["count", "mean"]).reindex(compass)

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].bar(wind.index, wind["count"], color="slategray")
axes[0].set_title("Frequency of Wind Directions (hours)")
axes[1].bar(wind.index, wind["mean"], color="indianred")
axes[1].set_title("Mean PM2.5 by Wind Direction")
axes[1].set_ylabel("Mean PM2.5 (µg/m³)")
for ax in axes:
    ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```

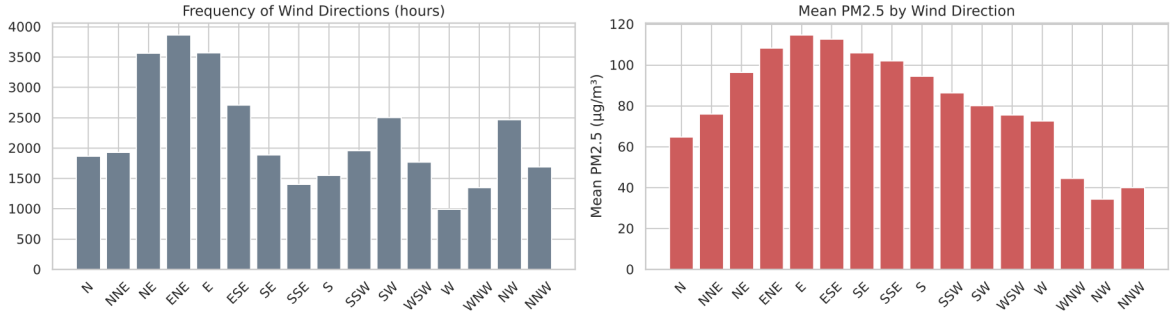


Figure 19: Frequency of wind directions and mean PM2.5 by direction

The most frequent winds are from the east-north-east, east and north-east. Wind direction has a very strong effect on pollution: mean PM2.5 is highest with easterly and south-easterly winds (114.8 µg/m³ for E and 112.7 for ESE) and lowest with north-westerly winds (34.4 for NW, 39.9 for NNW and 44.6 for WNW), a difference of more than three times. North-westerly winds bring clean, dry air from the sparsely populated Mongolian plateau, while easterly and southerly winds carry air from the densely populated and industrial North China Plain.

7.4 Correlation between variables

```
[19]: corr = df[["PM2.5", "PM10", "SO2", "NO2", "CO", "O3",
                 "TEMP", "PRES", "DEWP", "RAIN", "WSPM"]].corr()
      plt.figure(figsize=(9, 7))
      sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
      plt.title("Correlation Matrix (Pearson, hourly data)")
      plt.show()
```

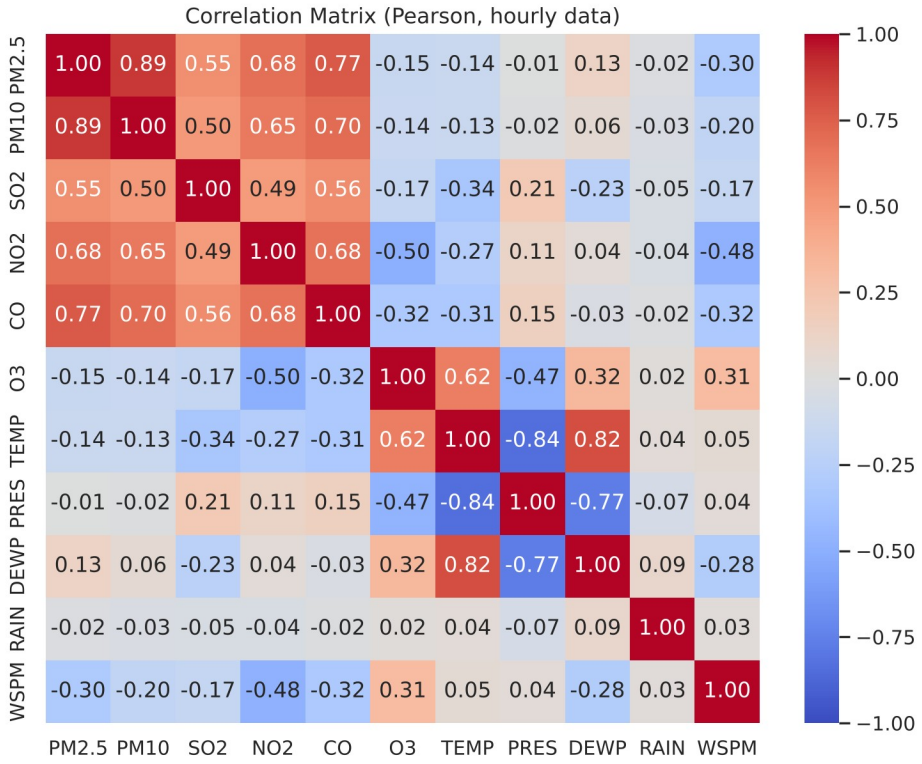


Figure 20: Pearson correlation matrix of the numerical variables

PM2.5 is very strongly correlated with PM10 (0.89) and strongly with CO (0.77) and NO2 (0.68), because these pollutants share sources (combustion and traffic) and are trapped by the same weather. Wind speed has the clearest negative relationship with PM2.5 (−0.30), as wind disperses pollution. Ozone behaves differently from the other pollutants: it is positively correlated with temperature (0.62) and negatively with NO2 (−0.50), because ozone is produced by sunlight and consumed by NO2. The weather variables are strongly related to each other through the seasons (temperature with pressure −0.84 and with dew point 0.82). Rain shows almost no linear correlation with any variable because it rains in very few hours.

Temperature, pressure and dew point each have a weak linear correlation with PM2.5 (between −0.14 and 0.13). This does not mean that weather is unimportant; rather, its effect is non-linear and depends on combinations of conditions, which the next section examines.

8. Multivariate Analysis

```
[20]: plt.figure(figsize=(9, 5))
      sc = plt.scatter(daily["DEWP"], daily["PM2.5"], c=daily["WSPM"],
                       cmap="viridis_r", s=14, alpha=0.8)
      plt.colorbar(sc, label="Wind speed (m/s)")
      plt.title("Daily PM2.5 vs Dew Point, Coloured by Wind Speed")
      plt.xlabel("Dew point (°C)")
      plt.ylabel("PM2.5 (µg/m³)")
      plt.show()
```

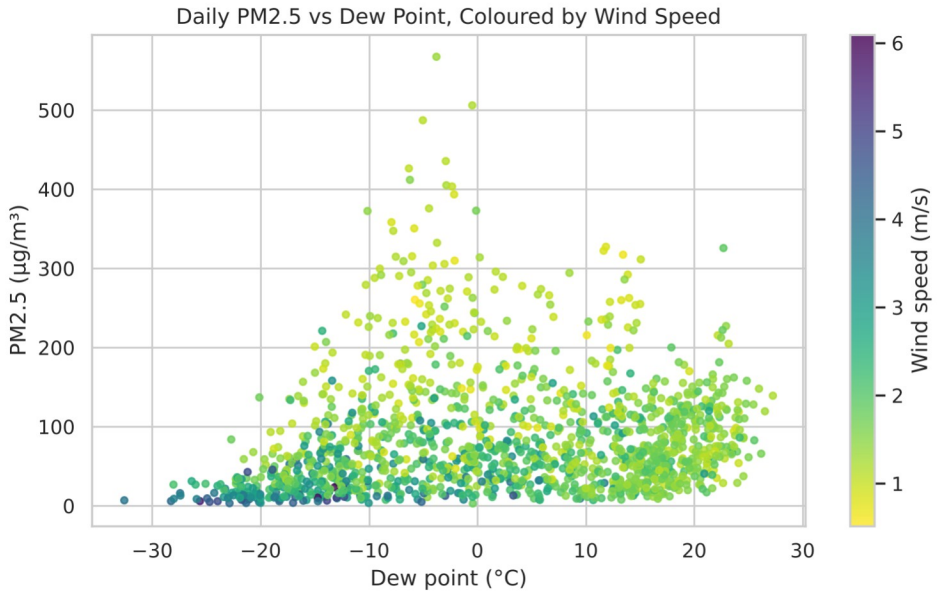


Figure 21: Daily PM2.5 against dew point, coloured by wind speed

This plot combines three variables. The polluted days (high on the y-axis) are almost all yellow, meaning calm conditions: 25 of the 28 days above 300 µg/m³ had an average wind speed below 1.5 m/s. Days with wind above 3 m/s average only 34.3 µg/m³, compared with 152.2 µg/m³ on days with wind below 1.2 m/s. Very dry days are almost always clean: of the 169 days with a dew point below −15 °C, 161 had PM2.5 below 100 µg/m³. These dry, windy days usually follow cold fronts from the north-west.

The combination of **calm air and moderate-to-high humidity** produces the worst haze, while wind speed alone separates clean from polluted days well. This is why the forecasting model in Section 10 uses wind speed and dew point.

9. Time Series Analysis

```
[21]: plt.figure(figsize=(13, 4))
      plt.plot(daily.index, daily["PM2.5"], color="lightsteelblue", label="Daily mean")
      plt.plot(daily.index, daily["PM2.5"].rolling(30).mean(), color="navy", label="30-day rolling mean")
      plt.title("Daily Mean PM2.5 at Dongsi (Mar 2013 - Feb 2017)")
      plt.ylabel("PM2.5 (µg/m³)")
      plt.legend()
      plt.show()
```

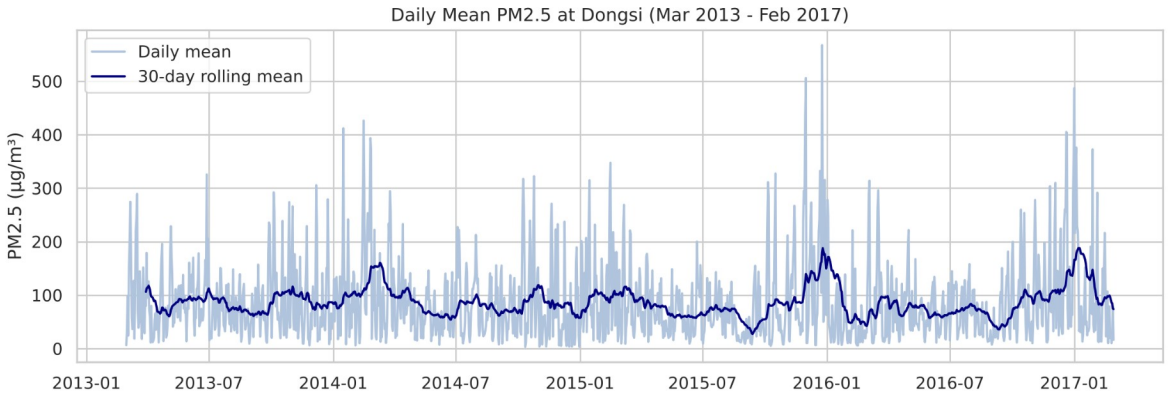


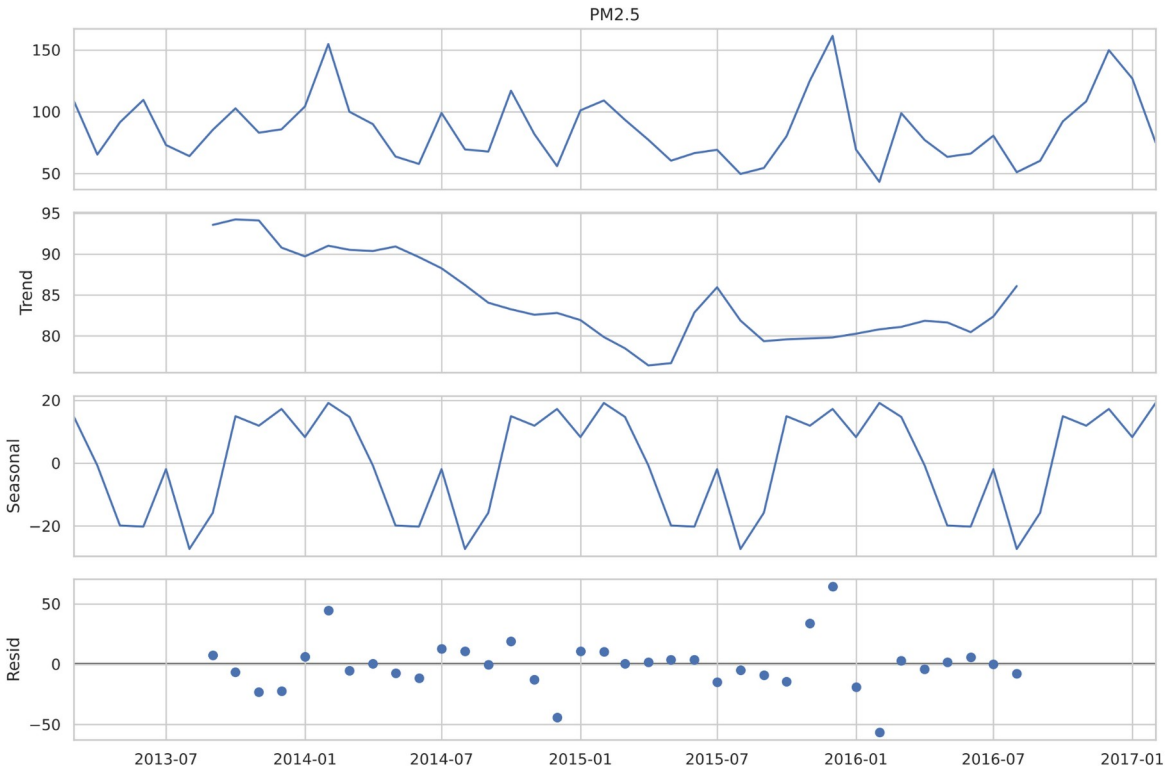
Figure 22: Daily mean PM2.5 with a 30-day rolling mean

Daily PM2.5 changes sharply from day to day, and the largest spikes all occur in late autumn and winter. The five worst days were 25 December 2015 (567.4 µg/m³), 1 December 2015 (506.0), 1 January 2017 (487.2), 30 November 2015 (435.6) and 15 February 2014 (426.3). The 30-day rolling mean rises every winter, peaking at about 188 µg/m³ in late December 2015 and early January 2017, whereas the winter of 2014–15 was milder and its rolling mean peaked at only 110. The cleanest period was late summer 2015, when the rolling mean fell to 27.3 µg/m³ in mid-September; this coincides with the emission controls imposed around Beijing for the military parade of 3 September 2015.

[22]:
```
from statsmodels.tsa.seasonal import seasonal_decompose

monthly = daily["PM2.5"].resample("MS").mean()
decomp = seasonal_decompose(monthly, model="additive", period=12)
fig = decomp.plot()
fig.set_size_inches(12, 8)
plt.tight_layout()
plt.show()

# seasonal effect for each calendar month (µg/m³ above / below the trend)
decomp.seasonal.groupby(decomp.seasonal.index.month).first().round(1)
```



[22]:
```
datetime
1      8.3
2     19.1
3     14.7
4     -0.8
5    -19.9
6    -20.2
7     -2.0
8    -27.3
9    -15.8
10    14.9
11    11.9
12    17.2
Name: seasonal, dtype: float64
```

Figure 23: Additive decomposition of monthly mean PM2.5 (period = 12)

Monthly averages are used because a daily series with only four years of data gives a very noisy yearly seasonal component. The seasonal component shows that PM2.5 is above the trend from October to March (+19.1 µg/m³ in February, +17.2 in December, +14.9 in October) and well below it from May to September (−27.3 in August, −20.2 in June, −19.9 in May). July is an exception at only −2.0, which is typical of humid, stagnant mid-summer weather.

The trend falls from about 94 µg/m³ in late 2013 to about 76 in spring 2015, then levels off at around 80–86 µg/m³ through 2016. The early decline is consistent with the national air-pollution action plan introduced in late 2013, but the improvement did not continue in 2016. The largest residuals fall in the cold months, for example December 2015 (+64.3 µg/m³) and February 2016 (−56.7), because the number of extreme episodes in a winter month varies a lot from year to year.

```
[23]: from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
plot_acf(daily["PM2.5"], lags=30, ax=axes[0])
plot_pacf(daily["PM2.5"], lags=30, ax=axes[1])
plt.tight_layout()
plt.show()
```

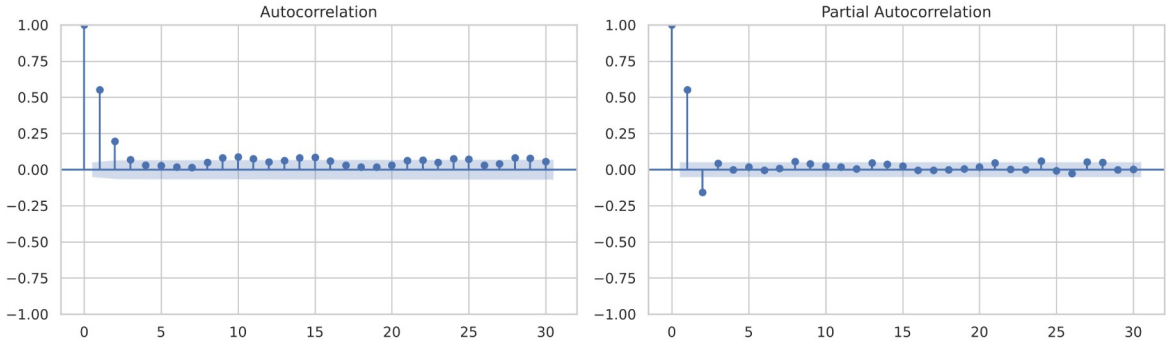


Figure 24: Autocorrelation (ACF) and partial autocorrelation (PACF) of daily PM2.5

The autocorrelation at lag 1 is 0.55 and at lag 2 only 0.20, after which it falls inside the confidence band. The PACF has one large spike at lag 1 (0.55) and a small negative one at lag 2 (−0.16). Today's PM2.5 therefore carries useful information about tomorrow, but this memory fades within about three days: pollution builds up over a few calm days and is then cleared away by the next change in the weather.

This has two consequences for modelling. Recent PM2.5 values should be used as features, but on their own they cannot predict when an episode will start or end, so information about the coming weather is needed.

10. Forecasting Model

10.1 Feature engineering and train/test split

[24]:
```
# target = tomorrow's daily mean PM2.5
data = pd.DataFrame(index=daily.index)
data["pm_today"] = daily["PM2.5"]
data["pm_yesterday"] = daily["PM2.5"].shift(1)
data["pm_7day_mean"] = daily["PM2.5"].rolling(7).mean()
data["month"] = daily.index.month
# tomorrow's weather (in practice taken from a weather forecast)
for col in ["TEMP", "PRES", "DEWP", "WSPM", "RAIN"]:
    data[col + "_tomorrow"] = daily[col].shift(-1)
data["target"] = daily["PM2.5"].shift(-1)
data = data.dropna()

# chronological split: last 12 months are kept for testing
train = data[data.index < "2016-03-01"]
test = data[data.index >= "2016-03-01"]
features = [c for c in data.columns if c != "target"]
print("Features:", features)
print("Train:", train.index.min().date(), "to", train.index.max().date(), "-", len(train), "days")
print("Test :", test.index.min().date(), "to", test.index.max().date(), "-", len(test), "days")
```

```
Features: ['pm_today', 'pm_yesterday', 'pm_7day_mean', 'month', 'TEMP_tomorrow', 'PRES_tomorrow', 'DEWP_tomorrow',
'WSPM_tomorrow', 'RAIN_tomorrow']
Train: 2013-03-07 to 2016-02-29 - 1090 days
Test : 2016-03-01 to 2017-02-27 - 364 days
```

Figure 25: Building the features and the chronological train/test split

The target is **tomorrow's daily mean PM2.5**. Based on the EDA, the features are today's and yesterday's PM2.5 and the 7-day average (the short memory found in the ACF), the month (seasonality), and tomorrow's temperature, pressure, dew point, wind speed and rainfall (the weather effects found in Sections 7 and 8).

Using tomorrow's weather assumes that a weather forecast is available, which is how air-quality forecasts are made in practice. Here the observed weather is used as a stand-in for that forecast, so real-world errors would be somewhat larger. The data are split by time, not randomly: 1,090 days (March 2013 to February 2016) for training and the final 364 days (March 2016 to February 2017) for testing. This mirrors real use, where the model only ever predicts the future.

10.2 Model training and evaluation

[25]:
```python
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def evaluate(name, y_true, y_pred):
    return {"model": name,
            "MAE": mean_absolute_error(y_true, y_pred),
            "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
            "R2": r2_score(y_true, y_pred)}

X_train, y_train = train[features], train["target"]
X_test, y_test = test[features], test["target"]

lr = LinearRegression().fit(X_train, y_train)
rf = RandomForestRegressor(n_estimators=300, min_samples_leaf=3, random_state=42)
rf.fit(X_train, y_train)

# same forest without tomorrow's weather, to measure how much the weather helps
history_only = ["pm_today", "pm_yesterday", "pm_7day_mean", "month"]
rf_hist = RandomForestRegressor(n_estimators=300, min_samples_leaf=3, random_state=42)
rf_hist.fit(train[history_only], y_train)

results = pd.DataFrame([
    evaluate("Baseline: tomorrow = today", y_test, test["pm_today"]),
    evaluate("Linear Regression", y_test, lr.predict(X_test)),
    evaluate("Random Forest", y_test, rf.predict(X_test)),
    evaluate("Random Forest (no weather)", y_test, rf_hist.predict(test[history_only])),
])
results.round(3)
```

[25]:

	model	MAE	RMSE	R2
0	Baseline: tomorrow = today	47.905	70.362	0.175
1	Linear Regression	36.990	52.031	0.549
2	Random Forest	31.213	43.566	0.684
3	Random Forest (no weather)	44.389	63.078	0.337

Figure 26: Training the models and comparing them on the test year

Three models are compared: a baseline that simply predicts tomorrow equals today, Linear Regression and a Random Forest with 300 trees. A fourth model, the same Random Forest without tomorrow's weather, measures how much the weather contributes.

Model	MAE (µg/m³)	RMSE (µg/m³)	R^2
Baseline: tomorrow = today	47.9	70.4	0.175
Linear Regression	37.0	52.0	0.549
Random Forest	31.2	43.6	0.684
Random Forest (no weather)	44.4	63.1	0.337

Table 2: Forecast accuracy on the test year (364 days)

The **Random Forest is the best model**, with a mean absolute error of 31.2 µg/m³ and R^2 of 0.684. Its error is 35% lower than the baseline's (47.9 µg/m³), and it explains about four times as much of the variation (R^2 0.684 compared with 0.175). It also outperforms Linear Regression (R^2 0.549) because it can capture the non-linear interaction between wind and humidity seen in Section 8. Removing tomorrow's weather halves its R^2 (0.684 to 0.337), which shows that the weather forecast is the key ingredient.

10.3 Forecast compared with actual values

```
[26]: pred = pd.Series(rf.predict(X_test), index=X_test.index)
forecast_day = y_test.index + pd.Timedelta(days=1)   # the day each forecast is for

plt.figure(figsize=(13, 4))
plt.plot(forecast_day, y_test, color="gray", label="Actual")
plt.plot(forecast_day, pred, color="crimson", label="Random Forest forecast")
plt.title("Next-Day PM2.5 Forecast on the Test Year (Mar 2016 - Feb 2017)")
plt.ylabel("PM2.5 (µg/m³)")
plt.legend()
plt.show()
```

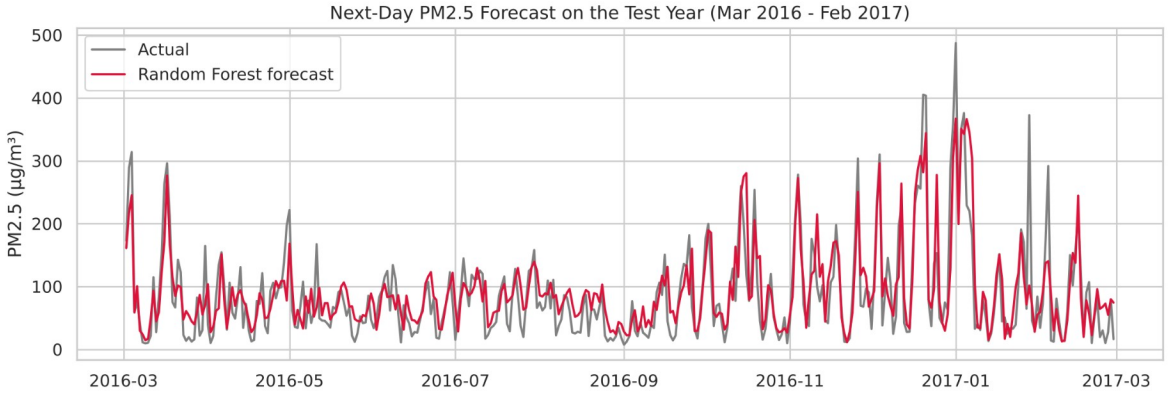


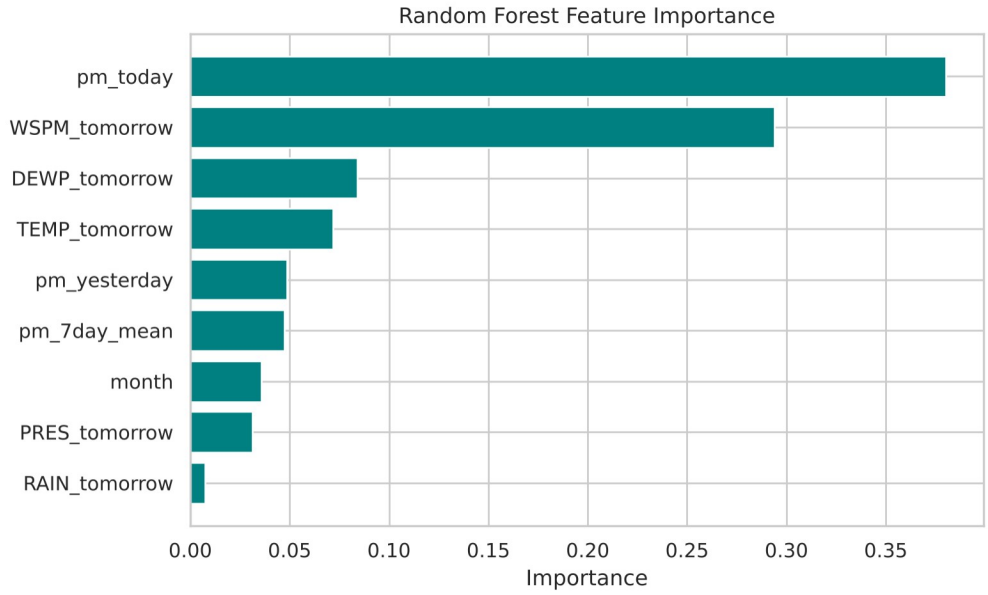
Figure 27: Random Forest next-day forecast against actual PM2.5 for the test year

The forecast follows the rise and fall of almost every pollution episode, including the heavy winter episodes of December 2016 and January 2017. Like most tree-based models, it tends to under-predict the very worst days: on 1 January 2017 the actual value was 487.2 µg/m³ against a forecast of 367.1, and on 20 December 2016 it was 405.1 against 281.6. The largest single miss was 28 January 2017 (actual 372.5, forecast 101.5), which was Chinese New Year: fireworks add a sudden burst of smoke that no weather variable can predict. Errors are larger in winter (mean absolute error 39.3 µg/m³ for December to February) than in summer (25.3 µg/m³ for June to August), simply because winter values are higher and more variable.

10.4 Feature importance

[27]:
```
importance = pd.Series(rf.feature_importances_, index=features).sort_values()
plt.figure(figsize=(8, 5))
plt.barh(importance.index, importance.values, color="teal")
plt.title("Random Forest Feature Importance")
plt.xlabel("Importance")
plt.show()

importance.sort_values(ascending=False).round(3)
```



[27]:
```
pm_today         0.380
WSPM_tomorrow    0.294
DEWP_tomorrow    0.084
TEMP_tomorrow    0.072
pm_yesterday     0.049
pm_7day_mean     0.047
month            0.036
PRES_tomorrow    0.031
RAIN_tomorrow    0.008
dtype: float64
```

Figure 28: Random Forest feature importance

Today's PM2.5 (0.380) and tomorrow's wind speed (0.294) together account for about two-thirds of the model's importance, followed by tomorrow's dew point (0.084) and temperature (0.072). This matches the EDA: pollution persists from one day to the next (ACF lag 1 = 0.55), wind is the main force that clears it, and humidity contributes to haze formation. Rainfall has almost no importance (0.008), consistent with its near-zero correlation with PM2.5.

10.5 Usefulness as a pollution warning

[28]:
```
# is the forecast useful as a warning? (China's 24-hour PM2.5 standard = 75 µg/m³)
warn = pd.DataFrame({"actual_above_75": y_test > 75, "forecast_above_75": pred > 75})
print(pd.crosstab(warn["actual_above_75"], warn["forecast_above_75"]), "\n")

hits = (warn["actual_above_75"] & warn["forecast_above_75"]).sum()
print(f"Polluted days correctly warned : {hits} of {warn['actual_above_75'].sum()}"
      f" ({hits / warn['actual_above_75'].sum():.1%})")

level_actual = pd.cut(y_test, bins=bins, labels=levels).cat.codes
level_pred = pd.cut(pred, bins=bins, labels=levels).cat.codes
print(f"AQI level exactly right        : {(level_actual == level_pred).mean():.1%}")
print(f"AQI level within one level     : {((level_actual - level_pred).abs() <= 1).mean():.1%}")
```

```
forecast_above_75  False  True
actual_above_75
False                146     57
True                  24    137

Polluted days correctly warned : 137 of 161 (85.1%)
AQI level exactly right        : 40.9%
AQI level within one level     : 90.4%
```

Figure 29: Evaluating the forecast as a warning for days above 75 µg/m³

In practice, people mainly want to know whether tomorrow will be polluted. Using China's 24-hour standard of 75 µg/m³ as the threshold, the model correctly warned of **137 of the 161 polluted days (85.1%)** in the test year and missed 24. It issued 57 false alarms out of 194 warnings (29%), which is an acceptable trade-off for a health warning, where missing a polluted day is worse than an unnecessary precaution. The forecast lands in exactly the right AQI level on 40.9% of days and within one level on 90.4% of days.

11. Key Findings

- **Data quality:** CO and NO2 had the most missing data (9.1% and 4.6%), with gaps of up to 63 days. 18 ozone readings of 413 to 1,071 µg/m³ were sensor errors and were removed. High PM2.5 values were real haze episodes and were kept.
- **Pollution is frequent:** 43.5% of days exceeded China's 24-hour PM2.5 standard of 75 µg/m³, and 14.2% were Heavily or Severely polluted.
- **Distribution:** PM2.5 is strongly right-skewed (mean 86.1, median 61, skewness 1.99). Most days are moderate, with occasional extreme episodes.
- **Season:** winter has the highest mean (103.1 µg/m³) and the most extreme days, but a similar median to other seasons. The yearly seasonal effect ranges from +19 µg/m³ in February to −27 in August.
- **Time of day:** in winter and autumn PM2.5 peaks at night (128 µg/m³ at 23:00 in winter) and is lowest in late morning.
- **Weather:** north-westerly winds give the cleanest air (34 µg/m³) and easterly winds the dirtiest (115 µg/m³). Calm days average 152 µg/m³ against 34 µg/m³ on windy days.
- **Short memory:** daily PM2.5 has a lag-1 autocorrelation of 0.55 that fades within about three days.
- **Forecasting:** a Random Forest using today's PM2.5 and tomorrow's weather achieved MAE 31.2 µg/m³ and R^2 0.684, compared with R^2 0.175 for the baseline, and correctly warned of 85.1% of polluted days.

12. Conclusion, Limitations and Future Work

12.1 Conclusion

This project carried out a complete exploratory data analysis of four years of hourly air-quality data from the Dongsi station in Beijing and used its findings to build a next-day PM2.5 forecast. The EDA showed that the data required careful cleaning: invalid ozone readings had to be identified from their physical impossibility, and missing values were filled in time rather than with a global average. It also showed that PM2.5 at this station is driven by the season, the time of day and above all the weather, with calm, humid conditions producing the worst haze and north-westerly winds clearing it.

These findings translated directly into the model design. Features chosen from the EDA, recent PM2.5 and tomorrow's wind, humidity and temperature, allowed a Random Forest to explain about 68% of the day-to-day variation in PM2.5 and to warn of 85% of polluted days, a large improvement over simply assuming that tomorrow will be like today.

12.2 Limitations

- Observed weather was used in place of a real weather forecast, so the reported accuracy is an upper limit; with forecast weather the errors would be somewhat larger.
- Only one of the 12 stations was analysed, so the results describe central Beijing and may differ at suburban sites.
- The long gaps in CO (63 days) and NO2 (37.5 days) were filled by linear interpolation, which smooths those periods.
- The model cannot anticipate events such as fireworks at Chinese New Year, as shown by its largest error.

12.3 Future work

- Use archived weather-forecast data instead of observations to measure realistic accuracy.
- Add a public-holiday indicator so the model can learn the effect of festival fireworks.
- Include data from neighbouring stations, since pollution often arrives from upwind sites first.
- Try other models such as gradient boosting, or forecast the AQI level directly as a classification problem.

References

[1] Chen, S. (2017). Beijing Multi-Site Air Quality [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5RK5G

[2] sid321axn. Beijing Multi-Site Air-Quality Data Set. Kaggle. https://www.kaggle.com/datasets/sid321axn/beijing-multisite-airquality-data-set

[3] Zhang, S., Guo, B., Dong, A., He, J., Xu, Z., & Chen, S. X. (2017). Cautionary tales on air-quality improvement in Beijing. Proceedings of the Royal Society A.

[4] Ministry of Environmental Protection of China (2012). HJ 633-2012: Technical Regulation on Ambient Air Quality Index (on trial).

[5] Ministry of Environmental Protection of China (2012). GB 3095-2012: Ambient Air Quality Standards.

[6] World Health Organization (2021). WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide.

[7] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

[8] Seabold, S., & Perktold, J. (2010). statsmodels: Econometric and Statistical Modeling with Python. Proceedings of the 9th Python in Science Conference.

[9] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Appendix A: Complete Python Code

The complete code of the notebook EDA_Theory_Project.ipynb, in the order the cells were run. Each block is marked with its cell number.

```
# ---- Cell [1] ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

sns.set_theme(style="whitegrid")
%config InlineBackend.figure_format = 'retina'

# ---- Cell [2] ----
df = pd.read_csv("PRSA_Data_Dongsi_20130301-20170228.csv")
print("Rows:", df.shape[0], " Columns:", df.shape[1])
df.head()

# ---- Cell [3] ----
df.info()

# ---- Cell [4] ----
print("Stations in file:", df["station"].unique().tolist())

# build one datetime column from year, month, day and hour
df["datetime"] = pd.to_datetime(df[["year", "month", "day", "hour"]])
df = df.set_index("datetime")
df = df.drop(columns=["No", "year", "month", "day", "hour", "station"])

# check duplicates and whether every hour is present
full_range = pd.date_range(df.index.min(), df.index.max(), freq="h")
print("Duplicate rows       :", df.duplicated().sum())
print("Duplicate timestamps :", df.index.duplicated().sum())
print("Expected hours       :", len(full_range), " Rows present:", len(df))

# ---- Cell [5] ----
def longest_gap(s):
    # longest run of consecutive missing hours in a column
    runs = s.isna().astype(int).groupby(s.notna().cumsum()).sum()
    return runs.max()

missing = pd.DataFrame({
    "missing_count": df.isna().sum(),
    "missing_pct": (df.isna().mean() * 100).round(2),
    "longest_gap_hours": [longest_gap(df[c]) for c in df.columns]
})
missing.sort_values("missing_pct", ascending=False)

# ---- Cell [6] ----
miss = missing[missing["missing_count"] > 0].sort_values("missing_pct")
plt.figure(figsize=(9, 4))
plt.barh(miss.index, miss["missing_pct"], color="indianred")
for i, v in enumerate(miss["missing_pct"]):
    plt.text(v + 0.1, i, f"{v}%", va="center", fontsize=9)
plt.title("Percentage of Missing Values per Column")
plt.xlabel("Missing (%)")
plt.show()
```

```
# ---- Cell [7] ----
pollutants = ["PM2.5", "PM10", "SO2", "NO2", "CO", "O3"]
fig, axes = plt.subplots(1, 6, figsize=(15, 4))
for ax, col in zip(axes, pollutants):
    sns.boxplot(y=df[col], ax=ax, color="lightsteelblue")
    ax.set_title(col)
    ax.set_ylabel("")
fig.suptitle("Box Plots of Pollutant Concentrations (µg/m³)")
plt.tight_layout()
plt.show()

# ---- Cell [8] ----
rows = []
for col in pollutants:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    upper = q3 + 1.5 * iqr
    rows.append([col, q1, q3, iqr, upper, (df[col] > upper).sum(), df[col].max()])

pd.DataFrame(rows, columns=["variable", "Q1", "Q3", "IQR", "upper_fence",
                            "n_outliers", "max"]).round(1)

# ---- Cell [9] ----
high_o3 = df[df["O3"] > 400]
print("O3 readings above 400:", len(high_o3))
print("Dates:", sorted(set(high_o3.index.date)))
df.loc["2013-08-15 20:00":"2013-08-16 03:00", ["NO2", "O3", "TEMP"]]

# ---- Cell [10] ----
# O3 jumps between 2 and 1071 within a few hours -> sensor error, mark as missing
df.loc[df["O3"] > 400, "O3"] = np.nan
print("O3 maximum after removing invalid readings:", df["O3"].max())

# ---- Cell [11] ----
num_cols = df.select_dtypes("number").columns
pm_before = df["PM2.5"].describe()

# numeric columns: time-based linear interpolation between neighbouring hours
df[num_cols] = df[num_cols].interpolate(method="time")
# wind direction (categorical): carry the last observed direction forward
df["wd"] = df["wd"].ffill()

print("Missing values left:", df.isna().sum().sum())
pd.DataFrame({"PM2.5 before": pm_before, "PM2.5 after": df["PM2.5"].describe()}).round(2)

# ---- Cell [12] ----
key = ["PM2.5", "PM10", "NO2", "CO", "O3", "TEMP", "DEWP", "WSPM"]
stats = df[key].describe().T
stats["skewness"] = df[key].skew()
stats["kurtosis"] = df[key].kurt()
stats.round(2)

# ---- Cell [13] ----
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
sns.histplot(df["PM2.5"], bins=60, kde=True, ax=axes[0], color="steelblue")
axes[0].set_title("Hourly PM2.5")
axes[0].set_xlabel("PM2.5 (µg/m³)")
sns.histplot(np.log1p(df["PM2.5"]), bins=60, kde=True, ax=axes[1], color="darkorange")
axes[1].set_title("log(1 + PM2.5)")
axes[1].set_xlabel("log(1 + PM2.5)")
```

```
plt.tight_layout()
plt.show()

# ---- Cell [14] ----
# daily average and China's AQI levels for 24-hour PM2.5
daily = df.resample("D").mean(numeric_only=True)

bins = [0, 35, 75, 115, 150, 250, np.inf]
levels = ["Excellent", "Good", "Lightly polluted", "Moderately polluted",
          "Heavily polluted", "Severely polluted"]
daily["aqi_level"] = pd.cut(daily["PM2.5"], bins=bins, labels=levels)

freq = daily["aqi_level"].value_counts().reindex(levels)
aqi_table = pd.DataFrame({"days": freq, "percent": (freq / len(daily) * 100).round(1)})
print("Number of days:", len(daily))
aqi_table

# ---- Cell [15] ----
colors = ["#2e7d32", "#9ccc65", "#fdd835", "#fb8c00", "#e53935", "#6a1b9a"]
plt.figure(figsize=(10, 4))
bars = plt.bar(aqi_table.index, aqi_table["days"], color=colors)
plt.bar_label(bars, labels=[f"{p}%" for p in aqi_table["percent"]])
plt.title("Number of Days in Each AQI Level (Daily Mean PM2.5)")
plt.ylabel("Days")
plt.xticks(rotation=15)
plt.show()

# ---- Cell [16] ----
season_map = {12: "Winter", 1: "Winter", 2: "Winter", 3: "Spring", 4: "Spring", 5: "Spring",
              6: "Summer", 7: "Summer", 8: "Summer", 9: "Autumn", 10: "Autumn", 11: "Autumn"}
daily["month"] = daily.index.month
daily["season"] = daily["month"].map(season_map)

plt.figure(figsize=(11, 4))
sns.boxplot(data=daily, x="month", y="PM2.5", color="lightsteelblue")
plt.title("Daily Mean PM2.5 by Month")
plt.xlabel("Month")
plt.ylabel("PM2.5 (μg/m³)")
plt.show()

daily.groupby("season")["PM2.5"].agg(["mean", "median", "std", "max"]).round(1)

# ---- Cell [17] ----
df["hour"] = df.index.hour
df["season"] = df.index.month.map(season_map)

hourly_profile = df.groupby(["season", "hour"])["PM2.5"].mean().reset_index()
plt.figure(figsize=(10, 4))
sns.lineplot(data=hourly_profile, x="hour", y="PM2.5", hue="season",
             hue_order=["Winter", "Spring", "Summer", "Autumn"], marker="o")
plt.title("Average PM2.5 by Hour of Day and Season")
plt.xlabel("Hour of day")
plt.ylabel("Mean PM2.5 (μg/m³)")
plt.xticks(range(0, 24, 2))
plt.show()

# ---- Cell [18] ----
compass = ["N", "NNE", "NE", "ENE", "E", "ESE", "SE", "SSE",
           "S", "SSW", "SW", "WSW", "W", "WNW", "NW", "NNW"]
wind = df.groupby("wd")["PM2.5"].agg(["count", "mean"]).reindex(compass)
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].bar(wind.index, wind["count"], color="slategray")
axes[0].set_title("Frequency of Wind Directions (hours)")
axes[1].bar(wind.index, wind["mean"], color="indianred")
axes[1].set_title("Mean PM2.5 by Wind Direction")
axes[1].set_ylabel("Mean PM2.5 (µg/m³)")
for ax in axes:
    ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()

# ---- Cell [19] ----
corr = df[["PM2.5", "PM10", "SO2", "NO2", "CO", "O3",
           "TEMP", "PRES", "DEWP", "RAIN", "WSPM"]].corr()
plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation Matrix (Pearson, hourly data)")
plt.show()

# ---- Cell [20] ----
plt.figure(figsize=(9, 5))
sc = plt.scatter(daily["DEWP"], daily["PM2.5"], c=daily["WSPM"],
                 cmap="viridis_r", s=14, alpha=0.8)
plt.colorbar(sc, label="Wind speed (m/s)")
plt.title("Daily PM2.5 vs Dew Point, Coloured by Wind Speed")
plt.xlabel("Dew point (°C)")
plt.ylabel("PM2.5 (µg/m³)")
plt.show()

# ---- Cell [21] ----
plt.figure(figsize=(13, 4))
plt.plot(daily.index, daily["PM2.5"], color="lightsteelblue", label="Daily mean")
plt.plot(daily.index, daily["PM2.5"].rolling(30).mean(), color="navy", label="30-day rolling
mean")
plt.title("Daily Mean PM2.5 at Dongsi (Mar 2013 - Feb 2017)")
plt.ylabel("PM2.5 (µg/m³)")
plt.legend()
plt.show()

# ---- Cell [22] ----
from statsmodels.tsa.seasonal import seasonal_decompose

monthly = daily["PM2.5"].resample("MS").mean()
decomp = seasonal_decompose(monthly, model="additive", period=12)
fig = decomp.plot()
fig.set_size_inches(12, 8)
plt.tight_layout()
plt.show()

# seasonal effect for each calendar month (µg/m³ above / below the trend)
decomp.seasonal.groupby(decomp.seasonal.index.month).first().round(1)

# ---- Cell [23] ----
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
plot_acf(daily["PM2.5"], lags=30, ax=axes[0])
plot_pacf(daily["PM2.5"], lags=30, ax=axes[1])
plt.tight_layout()
plt.show()
```

```
# ---- Cell [24] ----
# target = tomorrow's daily mean PM2.5
data = pd.DataFrame(index=daily.index)
data["pm_today"] = daily["PM2.5"]
data["pm_yesterday"] = daily["PM2.5"].shift(1)
data["pm_7day_mean"] = daily["PM2.5"].rolling(7).mean()
data["month"] = daily.index.month
# tomorrow's weather (in practice taken from a weather forecast)
for col in ["TEMP", "PRES", "DEWP", "WSPM", "RAIN"]:
    data[col + "_tomorrow"] = daily[col].shift(-1)
data["target"] = daily["PM2.5"].shift(-1)
data = data.dropna()

# chronological split: last 12 months are kept for testing
train = data[data.index < "2016-03-01"]
test = data[data.index >= "2016-03-01"]
features = [c for c in data.columns if c != "target"]
print("Features:", features)
print("Train:", train.index.min().date(), "to", train.index.max().date(), "-", len(train),
"days")
print("Test :", test.index.min().date(), "to", test.index.max().date(), "-", len(test), "days")

# ---- Cell [25] ----
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def evaluate(name, y_true, y_pred):
    return {"model": name,
            "MAE": mean_absolute_error(y_true, y_pred),
            "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
            "R2": r2_score(y_true, y_pred)}

X_train, y_train = train[features], train["target"]
X_test, y_test = test[features], test["target"]

lr = LinearRegression().fit(X_train, y_train)
rf = RandomForestRegressor(n_estimators=300, min_samples_leaf=3, random_state=42)
rf.fit(X_train, y_train)

# same forest without tomorrow's weather, to measure how much the weather helps
history_only = ["pm_today", "pm_yesterday", "pm_7day_mean", "month"]
rf_hist = RandomForestRegressor(n_estimators=300, min_samples_leaf=3, random_state=42)
rf_hist.fit(train[history_only], y_train)

results = pd.DataFrame([
    evaluate("Baseline: tomorrow = today", y_test, test["pm_today"]),
    evaluate("Linear Regression", y_test, lr.predict(X_test)),
    evaluate("Random Forest", y_test, rf.predict(X_test)),
    evaluate("Random Forest (no weather)", y_test, rf_hist.predict(test[history_only])),
])
results.round(3)

# ---- Cell [26] ----
pred = pd.Series(rf.predict(X_test), index=X_test.index)
forecast_day = y_test.index + pd.Timedelta(days=1)   # the day each forecast is for

plt.figure(figsize=(13, 4))
plt.plot(forecast_day, y_test, color="gray", label="Actual")
plt.plot(forecast_day, pred, color="crimson", label="Random Forest forecast")
plt.title("Next-Day PM2.5 Forecast on the Test Year (Mar 2016 - Feb 2017)")
```

```
plt.ylabel("PM2.5 (μg/m³)")
plt.legend()
plt.show()

# ---- Cell [27] ----
importance = pd.Series(rf.feature_importances_, index=features).sort_values()
plt.figure(figsize=(8, 5))
plt.barh(importance.index, importance.values, color="teal")
plt.title("Random Forest Feature Importance")
plt.xlabel("Importance")
plt.show()

importance.sort_values(ascending=False).round(3)

# ---- Cell [28] ----
# is the forecast useful as a warning? (China's 24-hour PM2.5 standard = 75 μg/m³)
warn = pd.DataFrame({"actual_above_75": y_test > 75, "forecast_above_75": pred > 75})
print(pd.crosstab(warn["actual_above_75"], warn["forecast_above_75"]), "\n")

hits = (warn["actual_above_75"] & warn["forecast_above_75"]).sum()
print(f"Polluted days correctly warned : {hits} of {warn['actual_above_75'].sum()}"
      f" ({hits / warn['actual_above_75'].sum():.1%})")

level_actual = pd.cut(y_test, bins=bins, labels=levels).cat.codes
level_pred = pd.cut(pred, bins=bins, labels=levels).cat.codes
print(f"AQI level exactly right        : {(level_actual == level_pred).mean():.1%}")
print(f"AQI level within one level     : {((level_actual - level_pred).abs() <=
1).mean():.1%}")
```